PYTHON CONCURRENCY DEEP DIVE

1. THE GLOBAL INTERPRETER LOCK (GIL)

In CPython, the Global Interpreter Lock (GIL) is a mutex that prevents multiple native threads from executing Python bytecodes at once. This lock is necessary because CPython's memory management is not thread-safe (reference counting).

2. CONCURRENCY PARADIGMS COMPARED

- Multiprocessing: Bypasses GIL by creating separate OS processes with dedicated memory heaps. Ideal for CPU-bound computations like matrix multiplication or image processing.
- Threading: Uses OS threads sharing the same memory heap. Best suited for I/O-bound tasks like downloading web pages or querying relational databases where threads release the GIL.
- Asyncio: Single-threaded cooperative multitasking utilizing an event loop and coroutines (async/await). Massive scalability for concurrent network sockets and WebSockets.

3. PRO TIPS FOR BACKEND ENGINEERS

Use uvloop as an event loop drop-in replacement to boost asyncio throughput by 2x-4x. Always guard against blocking calls inside async handlers by offloading to loop.run_in_executor().

ASYNCIO INTERNALS & COMMON PITFALLS

1. EVENT LOOP STARVATION
A synchronous time.sleep() or CPU heavy loop in a coroutine freezes the entire event loop, preventing all other concurrent connections from progressing.

2. TASK CANCELATION & CLEANUP
Always handle asyncio.CancelledError when wrapping tasks in asyncio.shield() or cleanup contexts to ensure open database connections and files are properly closed.

3. INTERVIEW FLASHCARD HIGHLIGHTS
- Q: What makes asyncio lightweight compared to threads?
- A: Coroutines do not require OS context switches or heavy thread stack allocations (typically only few hundred bytes).
- Q: When should ProcessPoolExecutor be preferred over ThreadPoolExecutor?
- A: Whenever the workload is CPU-bound and requires multiple CPU cores.